

bits válido
ventana(s)
instrucciones
o
estación o
estaciones
de reserva
(al finalizar
el ciclo)

	x86-64	Comentarios	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16	17
1	mul rax, rdx	rax <- rax*rdx																	
2	add rax, r9	rax <- rax+r9																	
3	shl rax, 3	rax <- rax << 3																	
4	mov rbx, r9	rbx <- r9																	
5	add r9,2	r9 <- r9 +2																	
6	shl rbx, 3	rbx <- rbx << 3																	
7	movsd xmm1, [rdi+rax]	xmm1 <- M[rdi+rax]																	
8	movsd xmm2, [rsi+rbx]	xmm2 <- M[rsi+rbx]																	
9	movsd xmm3, 8[rdi+rax]	xmm3 <- M[rdi+rax+8]																	
10	movsd xmm4, 8[rsi+rbx]	xmm4 <- M[rsi+rbx+8]																	
11	mulsd xmm1, xmm2	xmm1 <- xmm1 * xmm2																	
12	mulsd xmm3, xmm4	xmm3 <- xmm3 * xmm4																	
13	addsd xmm0, xmm1	xmm0 <- xmm0 + xmm1																	
14	addsd xmm0, xmm3	xmm0 <- xmm0 + xmm3																	
	x86-64	Comentarios	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16	17
1	mul rax, rdx	rax <- rax*rdx	11																
2	add rax, r9	rax <- rax+r9	01																
3	shl rax, 3	rax <- rax << 3	0																
4	mov rbx, r9	rbx <- r9	1																
5	add r9,2	r9 <- r9 +2	1																
6	shl rbx, 3	rbx <- rbx << 3	0																
7	movsd xmm1, [rdi+rax]	xmm1 <- M[rdi+rax]	10																
8	movsd xmm2, [rsi+rbx]	xmm2 <- M[rsi+rbx]	10																
9	movsd xmm3, 8[rdi+rax]	xmm3 <- M[rdi+rax+8]	10																
10	movsd xmm4, 8[rsi+rbx]	xmm4 <- M[rsi+rbx+8]	10																
11	mulsd xmm1, xmm2	xmm1 <- xmm1 * xmm2	00																
12	mulsd xmm3, xmm4	xmm3 <- xmm3 * xmm4	00																
13	addsd xmm0, xmm1	xmm0 <- xmm0 + xmm1	10																
14	addsd xmm0, xmm3	xmm0 <- xmm0 + xmm3	00																

El ensamblador utiliza notación de Intel (que no es la notación usada en Linux): el destino es el primer operando de la instrucción.

movsd xmm3, 8[rdi+rax]	xmm3 <- M[rdi+rax+8]	Se carga en el registro xmm3 el double que se encuentra almacenado a partir de la dirección que se obtiene de la suma rdi+rax+8 (es una lectura/carga de memoria). Tiene 2 bits de válido porque usa dos registros (rdi y rax) para calcular la dirección de acceso. Inicialmente, rdi está disponible (bit válido 1), rax no está disponible (bit válido 0)
mov rbx, r9	rbx <- r9	Mueve el contenido de r9 a rbx. Inicialmente r9 está disponible (bit válido 1)

Posible pregunta en el control : ¿Qué instrucciones se están ejecutando en el ciclo 6?

Para contestar a las preguntas hay que rellenar la tabla primero.